

Create authentication service that returns JWT

pom.xml

```
<!-- Spring Security -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>

<!-- JWT -->
<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-api</artifactId>
    <version>0.11.5</version>
</dependency>

<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-impl</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>

<dependency>
    <groupId>io.jsonwebtoken</groupId>
    <artifactId>jjwt-jackson</artifactId>
    <version>0.11.5</version>
    <scope>runtime</scope>
</dependency>
```

application.properties

```
spring.application.name=spring-learn
server.port=8090
```

AuthenticationController.java

```
package com.cognizant.spring_learn.controller;

import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.SignatureAlgorithm;
import io.jsonwebtoken.security.Keys;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestHeader;
import org.springframework.web.bind.annotation.RestController;

import javax.crypto.SecretKey;
```

```

import java.nio.charset.StandardCharsets;
import java.util.Base64;
import java.util.Date;
import java.util.HashMap;
import java.util.Map;

@RestController
public class AuthenticationController {

    private static final String SECRET =
        "mysecretkeymysecretkeymysecretkey12";

    @GetMapping("/authenticate")
    public Map<String, String> authenticate(
        @RequestHeader("Authorization") String authHeader) {

        String encoded = authHeader.substring(6);

        String credentials =
            new String(Base64.getDecoder().decode(encoded));

        String user = credentials.split(":")[0];

        SecretKey key =
            Keys.hmacShaKeyFor(SECRET.getBytes(StandardCharsets.UTF_8));

        String token = Jwts.builder()
            .setSubject(user)
            .setIssuedAt(new Date())
            .setExpiration(new Date(System.currentTimeMillis() + 1200000))
            .signWith(key, SignatureAlgorithm.HS256)
            .compact();

        Map<String, String> map = new HashMap<>();
        map.put("token", token);

        return map;
    }
}

```

JwtAuthorizationFilter.java

```

package com.cognizant.spring_learn.security;

import io.jsonwebtoken.Claims;
import io.jsonwebtoken.Jwts;
import io.jsonwebtoken.security.Keys;

```

```
import jakarta.servlet.FilterChain;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;
import org.springframework.security.authentication.UsernamePasswordAuthenticationToken;
import org.springframework.security.core.authority.SimpleGrantedAuthority;
import org.springframework.security.core.context.SecurityContextHolder;
import org.springframework.web.filter.OncePerRequestFilter;
```

```
import javax.crypto.SecretKey;
import java.io.IOException;
import java.nio.charset.StandardCharsets;
import java.util.List;
```

```
public class JwtAuthorizationFilter extends OncePerRequestFilter {
```

```
    private static final String SECRET =
        "mysecretkeymysecretkeymysecretkey12";
```

```
    @Override
```

```
    protected void doFilterInternal(HttpServletRequest request,
                                    HttpServletResponse response,
                                    FilterChain filterChain)
        throws ServletException, IOException {
```

```
        String header = request.getHeader("Authorization");
```

```
        if (header != null && header.startsWith("Bearer ")) {
```

```
            String token = header.substring(7);
```

```
            try {
```

```
                SecretKey key =
                    Keys.hmacShaKeyFor(SECRET.getBytes(StandardCharsets.UTF_8));
```

```
                Claims claims = Jwts.parserBuilder()
                    .setSigningKey(key)
                    .build()
                    .parseClaimsJws(token)
                    .getBody();
```

```
                UsernamePasswordAuthenticationToken authentication =
                    new UsernamePasswordAuthenticationToken(
                        claims.getSubject(),
                        null,
                        List.of(new SimpleGrantedAuthority("ROLE_USER"))
                    );
```

```

        SecurityContextHolder.getContext()
            .setAuthentication(authentication);

    } catch (Exception e) {

        SecurityContextHolder.clearContext();
    }
}

filterChain.doFilter(request, response);
}
}

```

SecurityConfig.java

```

package com.cognizant.spring_learn.security;

import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.core.userdetails.User;
import org.springframework.security.core.userdetails.UserDetails;
import org.springframework.security.provisioning.InMemoryUserDetailsManager;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;

@Configuration
public class SecurityConfig {

    @Bean
    public InMemoryUserDetailsManager userDetailsService() {

        UserDetails user = User.withDefaultPasswordEncoder()
            .username("user")
            .password("pwd")
            .roles("USER")
            .build();

        UserDetails admin = User.withDefaultPasswordEncoder()
            .username("admin")
            .password("pwd")
            .roles("ADMIN", "USER")
            .build();
    }
}

```

```

        return new InMemoryUserDetailsManager(user, admin);
    }

    @Bean
    public SecurityFilterChain securityFilterChain(HttpSecurity http)
        throws Exception {

        http
            .csrf(csrf -> csrf.disable())
            .sessionManagement(session ->
                session.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/authenticate").authenticated()
                .requestMatchers("/countries/**").authenticated()
                .requestMatchers("/hello").permitAll()
                .anyRequest().permitAll()
            )
            .httpBasic(Customizer.withDefaults())
            .addFilterBefore(new JwtAuthorizationFilter(),
                UsernamePasswordAuthenticationFilter.class);

        return http.build();
    }
}

```



GET http://localhost:8090/countries

Show Request

HTTP/1.1 200

> (Headers) ...Content-Type: application/json...

```
[
  {
    "code": "IN",
    "name": "India"
  },
  {
    "code": "US",
    "name": "United States"
  },
  {
    "code": "JP",
    "name": "Japan"
  }
]
```

Response file saved.

> [2026-07-05T205545.200.json](#)